

컴퓨터비전(AI응용)

고급비전응용_SSD

Single Shot Multi-Box Detector

- SSD 기본 개념
- Loss Function
- Default Box Matching

Single Shot Detection & Multi-scale Prediction



One Forward Pass (Single Shot)

Faster R-CNN과 달리 Region Proposal 단계가 없는 **1-Stage Detector**입니다.
이미지를 한 번만 통과(Forward Pass)시켜 객체의 위치(Localization)와 클래스(Classification)를 동시에 예측합니다.



Multi-scale Feature Maps

다양한 크기의 객체를 검출하기 위해 여러 해상도의 Feature Map을 사용합니다.
높은 해상도는 작은 객체, 낮은 해상도는 큰 객체를 담당하여 Dense한 예측을 수행합니다.



Default Boxes (Anchors)

각 Feature Map의 셀마다 고정된 크기와 비율을 가진 Default Box를 미리 정의합니다.
네트워크는 이 Box들의 오프셋(Offset)과 클래스 점수를 예측하여 정밀도를 높입니다.

SSD는 YOLO의 속도와 Faster R-CNN의 정확도를 결합하기 위해 설계되었으며,
특히 멀티스케일 구조를 통해 다양한 크기의 객체 검출 성능을 극대화했습니다.

Multi-scale Feature Maps & VGG-16 Backbone



VGG-16 기반 Backbone과 확장

VGG-16 모델을 기본으로 하되 FC 레이어를 Conv 레이어(fc6, fc7)로 변환하고, 추가적인 컨볼루션 레이어를 더하여 깊이 있는 특징 추출 구조를 형성합니다.



6단계 예측 Feature Maps

해상도가 점차 줄어드는 6개의 Feature Map에서 각각 객체를 검출합니다.

Conv4_3

38 × 38

Conv7

19 × 19

Conv8_2

10 × 10

Conv9_2

5 × 5

Conv10_2

3 × 3

Conv11_2

1 × 1



해상도와 객체 크기의 관계

높은 해상도(38×38)는 작은 객체의 세밀한 특징을 포착하고,
낮은 해상도(1×1)는 넓은 Receptive Field를 통해 큰 객체를 검출합니다.

💡 Feature Pyramid 구조의 이점

단일 해상도 맵만 사용하는 것보다 다양한 크기의 Feature Map을 활용함으로써 **Scale Invariance(크기 불변성)**를 확보하고 작은 물체부터 큰 물체까지 놓치지 않고 검출할 수 있습니다.

왜 Multi-scale 인가?

Scale-Variance 문제 해결 및 객체 크기별 최적화



작은 객체: 고해상도 Feature Map (38×38)

초기 레이어(Conv4_3)는 해상도가 높아 세밀한 위치 정보를 보존합니다.
작은 Receptive Field를 가지므로 좁은 영역의 디테일을 포착하여 **작은 객체 검출**에 유리합니다.



큰 객체: 저해상도 Feature Map (3×3, 1×1)

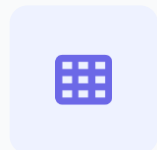
후반 레이어(Conv10_2, Conv11_2)는 풀링 등을 거쳐 해상도가 낮고 추상화 수준이 높습니다.
넓은 Receptive Field를 통해 전역 컨텍스트를 포착하여 **큰 객체 검출**에 최적화됩니다.



단일 해상도의 한계 극복

단일 해상도(특히 저해상도)만 사용 시 작은 객체의 정보가 소실되는 Scale-Variance 문제가 발생합니다.
SSD는 다중 해상도 피라미드를 통해 검출 누락과 오탐(False Positive)을 동시에 줄입니다.

Conv4_3 (38×38)



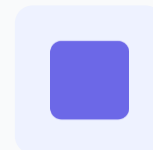
작은 객체, 세밀함

Conv7 (19×19)



중간 크기 객체

Conv11_2 (1×1)



큰 객체, 전체 맥락

✓ 핵심 원리

"다양한 크기의 Feature Map을 사용하여, 각 해상도에 맞는 크기의 객체를 전담하여 검출함으로써 정확도를 극대화합니다."

Default Boxes (Anchor Boxes)

스케일 및 종횡비(Aspect Ratio) 설정 메커니즘

스케일 계산 공식

$$s_k = s_{min} + \frac{s_{max} - s_{min}}{m - 1} (k - 1)$$

파라미터 정의

s_min	0.2 (최하위 레이어 스케일 20%)
s_max	0.9 (최상위 레이어 스케일 90%)
m	Feature Map의 총 개수 (SSD300 = 6)
k	현재 Feature Map의 인덱스 (1 ~ m)

i 낮은 레이어는 작은 객체, 높은 레이어는 큰 객체를 담당하도록 설계됨

종횡비 (Aspect Ratio)

각 Feature Map의 셀(Cell)마다 다양한 비율의 Box를 생성하여, 여러 형태의 객체에 대응합니다. (기본 4~6개)



매칭 전략 (Matching)

- ✓ Ground Truth와 IoU(Intersection over Union)가 높은 Default Box를 Positive로 매칭
- 🔍 매칭된 Default Box를 기준으로 Offset을 보정하고 클래스 점수를 예측

Feature Map별 Default Box 분포

SSD300 모델 기준 총 8,732개의 Default Box 구성

Feature Map	Resolution (H×W)	Boxes / Cell	Total Boxes
Conv4_3	38 × 38	4	5,776
Conv7	19 × 19	6	2,166
Conv8_2	10 × 10	6	600
Conv9_2	5 × 5	6	150
Conv10_2	3 × 3	6	54
Conv11_2	1 × 1	4	4
Total			8,732

분포 비율 분석



Conv4_3 (Small Obj) Conv7
Conv8_2 Others

💡 핵심 통찰:

전체 Box의 약 66%가 가장 높은 해상도인 **Conv4_3**에서 생성됩니다. 이는 작은 객체를 놓치지 않기 위해 고해상도 맵에 더 많은 앵커를 집중시킨 설계 의도입니다.

Classification, Localization & NMS

**분류 (Classification): C+1 클래스 확률**

각 Default Box마다 **C개의 객체 클래스와 1개의 배경 클래스**에 대한 점수를 예측합니다.
Softmax 함수를 통해 특정 클래스일 확률을 계산하며, 배경(Background) 클래스를 포함하여 객체가 없는 경우도 처리합니다.

**위치 회귀 (Localization): 4개의 오프셋**

Default Box가 실제 객체(Ground Truth)와 정확히 일치하도록 **4개의 오프셋 값**(Δcx , Δcy , Δw , Δh)을 예측합니다.
중심점의 이동량과 너비/높이의 비율 조절을 통해 박스를 미세 조정(Regression)합니다.

**후처리: NMS (Non-Maximum Suppression)**

추론 단계에서 수많은 중복 박스가 생성되므로, **신뢰도(Confidence)**가 가장 높은 박스만 남기고 나머지는 제거합니다.
IoU(겹침 정도)가 높은 박스들을 억제하여 최종적으로 하나의 객체당 하나의 박스만 출력합니다.

**예측 흐름 요약**

8,732개의 Default Box 생성 → CNN을 통한 분류 및 위치 오프셋 동시 예측 → NMS를 통한 중복 제거 → 최종 객체 검출 완료

YOLO vs SSD: 핵심 비교

단일 스케일(Single Scale)과 멀티 스케일(Multi-scale) 아키텍처의 차이



YOLO v1

You Only Look Once



Feature Map

마지막 단일 레이어 (7×7)만 사용



작은 객체 검출

저해상도로 인해 작은 물체에 **취약함**



Anchor Box

사용 안 함 (Grid Cell 직접 예측)



속도

매우 빠름 (Real-time 최적화)

VS



SSD

Single Shot MultiBox Detector



Feature Map

Pyramid 구조의 **Multi-scale (6개)** 활용



작은 객체 검출

고해상도 맵 활용으로 **성능 우수**



Default Box

다양한 비율의 Anchor Box 사용

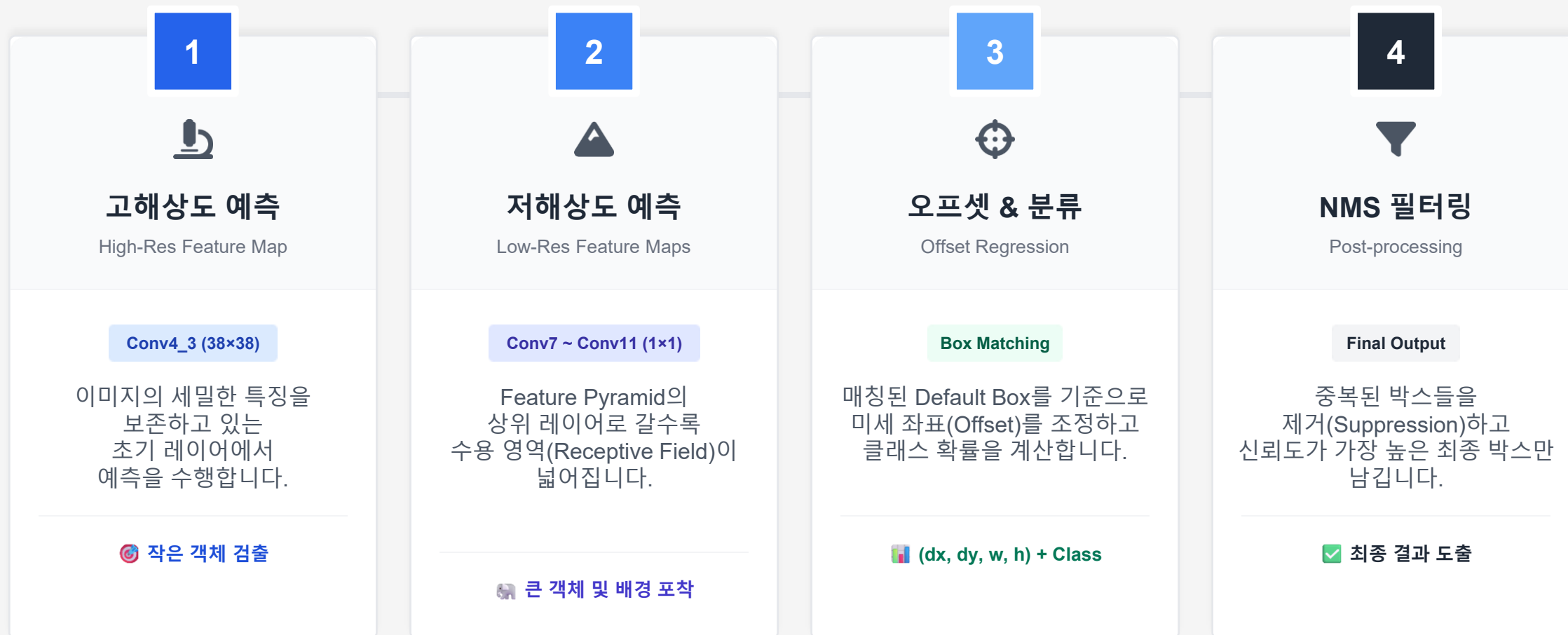


속도

빠름 (정확도와 속도의 균형)

SSD 검출 파이프라인

입력 이미지부터 최종 결과 도출까지의 4단계 프로세스



i SSD는 이 모든 과정을 단 한 번의 **Forward Pass** 내에서 수행하여 실시간(Real-time) 처리가 가능합니다.

Resolution Trade-off & Dense Prediction



SSD300 vs SSD512 (FPS 비교)

SSD300: 300×300 입력, 약 **59 FPS**로 실시간 처리에 최적화.

SSD512: 512×512 입력, 약 **22 FPS**로 속도는 느리지만 정확도가 높음.



Feature Maps Configuration

총 6개의 Scale (38×38, 19×19, 10×10, 5×5, 3×3, 1×1)을 사용하여 다양한 크기의 객체를 검출합니다.



압도적인 Default Boxes 수

SSD300 기준 총 **8,732개**의 Default Box를 사용합니다.

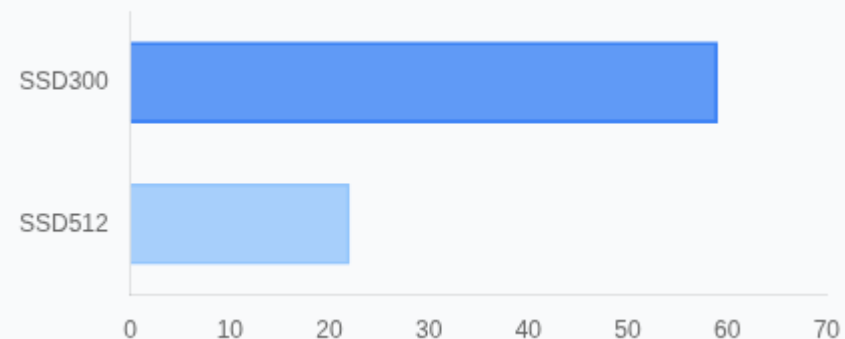
YOLO v1(98개) 대비 훨씬 조밀한(Dense) 예측을 수행하여 작은 객체 검출 성능을 강화했습니다.



핵심 결론

SSD는 입력 이미지 해상도 조절을 통해 **속도와 정확도의 밸런스**를 맞출 수 있으며, Multi-scale Feature Maps를 통해 **다양한 크기의 객체**를 효과적으로 커버합니다.

Input Resolution에 따른 FPS 비교



Default Boxes 수 비교 (vs YOLO v1)



SSD Loss Function 개요

Multi-task Loss: 분류(Classification)와 회귀(Regression)의 결합

전체 손실 함수 (Total Loss)

$$L(x, c, l, g) = \frac{1}{N} (L_{conf}(x, c) + \alpha L_{loc}(x, l, g))$$

- N : 매칭된 디폴트 박스(Default Box)의 개수입니다. 만약 $N = 0$ 이면 손실값은 0이 됩니다.
- x : 매칭 여부를 나타내는 지표 변수 (i 번째 박스가 j 클래스의 객체와 매칭되면 1, 아니면 0).
- c : 클래스 분류에 대한 예측 값.
- l : 예측된 경계 상자(Bounding Box)의 좌표.
- g : 실제 정답(Ground Truth)의 좌표.
- α : 두 손실 사이의 비중을 조절하는 가중치 매개변수 (기본값은 보통 1).

세부 구성 요소



Confidence Loss

객체 분류 오차 (Classification)

Softmax 함수를 사용하여 클래스 확률 계산

Cross Entropy Loss 적용

Positive(객체 있음)와 Negative(배경) 모두 고려

Hard Negative Mining을 통해 배경 클래스 불균형 해결



Localization Loss

위치 회귀 오차 (Bounding Box Regression)

Positive로 매칭된 Default Box에 대해서만 계산

Smooth L1 Loss 사용 (Outlier에 강건함)

중심 좌표(cx, cy)와 크기(w, h)의 Offset 예측

Ground Truth와의 차이를 최소화하도록 학습

Smooth L1 기반의 경계 상자(Bounding Box) 회귀



Positive Default Box 회귀 대상

Localization Loss는 오직 Ground Truth와 매칭된 **Positive Default Box**에 대해서만 계산됩니다.
예측 값은 절대 좌표가 아닌, Default Box의 중심좌표(cx, cy)와 크기(w, h)에 대한 **오프셋(Offset)**입니다.



Smooth L1 Loss 함수 적용

L1 Loss와 L2 Loss의 장점을 결합한 함수입니다.
오차가 작을 때는 **L2 (제곱)**처럼 동작하여 미분 가능성을 보장하고,
오차가 클 때는 **L1 (절대값)**처럼 동작하여 기울기가 일정하게 유지됩니다.



Outlier에 강건한(Robust) 학습

초기 학습 단계에서 발생할 수 있는 불안정한 예측값(Outlier)에 대해 **L2 Loss보다 덜 민감하게 반응**합니다.
이는 Gradient Explosion(기울기 폭주)을 방지하고 전체적인 학습 과정을 안정화시키는 핵심 역할을 합니다.



핵심 포인트

SSD는 Fast R-CNN에서 사용된 **Smooth L1 Loss**를 그대로 채택하여, 이상치에 영향을 덜 받으면서도 정밀한 위치 보정이 가능하도록 설계되었습니다.

SSD 좌표 파라미터화

학습을 위한 오프셋(Offset) 인코딩 수식

중심 좌표 오프셋 (Center)

$$\hat{g}^{cx} = \frac{g^{cx} - d^{cx}}{d^w \cdot variance[0]}$$

$$\hat{g}^{cy} = \frac{g^{cy} - d^{cy}}{d^h \cdot variance[1]}$$

핵심 원리: 절대 좌표를 직접 학습하지 않고, Default Box 크기(d_w, d_h)로 정규화된 상대적 거리(Offset)를 학습하여 위치 불변성(Translation Invariance)을 확보합니다.

파라미터 정의:

g Ground Truth (실제 정답 박스)

d Default Box (사전 정의된 앵커 박스)

cx, cy 박스의 중심 좌표 (Center X, Center Y)

크기 오프셋 (Size)

$$\hat{g}^w = \log \left(\frac{g^w}{d^w} \right) / variance[2]$$

$$\hat{g}^h = \log \left(\frac{g^h}{d^h} \right) / variance[3]$$

핵심 원리: 크기 비율에 로그를 취함으로써, 작은 박스와 큰 박스 간의 오차 스케일을 맞추는 다. 이는 다양한 크기의 객체를 동일한 비중으로 학습하는 **스케일 불변성(Scale Invariance)**을 돕습니다.

파라미터 정의:

w, h 박스의 너비(Width)와 높이(Height)

log 자연 로그 (Natural Logarithm)

g/d Ground Truth와 Default Box의 비율

Localization Loss: Smooth L1

안정적인 위치 회귀(Regression)를 위한 손실 함수 설계

수식 정의

$$\text{smooth}_{L1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

핵심 변수

x 예측값과 정답(GT) 간의 차이 (Error)

|x| < 1 오차가 작을 때: **L2 Loss** 처럼 동작
(0 근처에서 부드러운 미분)

|x| ≥ 1 오차가 클 때: **L1 Loss** 처럼 동작
(Gradient 폭주 방지)

L1 vs L2 vs Smooth L1 비교

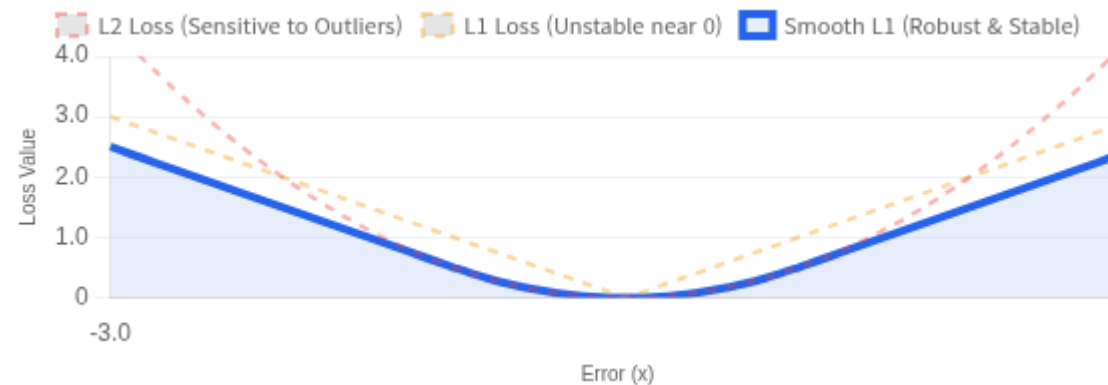
✖ L2 Loss (MSE)의 문제

오차가 클 때 제곱으로 패널티가 커져
Outlier에 매우 민감하며,
Gradient가 폭주할 수 있음.

! L1 Loss (MAE)의 문제

Outlier에는 강건하나,
0 근처에서 미분 불가능하거나
진동하여 수렴이 불안정함.

손실 함수 형태 비교 그래프



* Smooth L1은 초기(작은 오차)에는 부드럽게 수렴하고, 후기(큰 오차)에는 안정적으로 학습됨

Softmax Classifier & Cross Entropy



Softmax + Cross Entropy

각 Default Box에 대해 $C+1$ 개(객체 클래스 + 배경)의 클래스 점수를 예측합니다.
Softmax 함수를 사용하여 점수를 확률로 변환하고,
Cross Entropy Loss를 통해 정답 클래스의 확률을 최대화하도록 학습합니다.



Positive Examples (객체)

Ground Truth와 매칭된 Default Box들입니다. (주로 $IoU > 0.5$)
해당 객체의 클래스(c)에 대한 신뢰도 손실을 계산하여, 객체를 정확히 분류하도록 유도합니다.



Negative Examples (배경)

어떤 객체와도 매칭되지 않은 Default Box들입니다.
이들은 '배경(Background)' 클래스로 분류되어야 하며, 배경 클래스($c=0$)에 대한 손실을 계산합니다.



핵심 이슈: Class Imbalance (클래스 불균형)

전체 Default Box 중 배경(Negative)의 비율이 압도적으로 높습니다.
모든 Negative를 학습에 사용하면 모델이 배경만 예측하게 되는 문제가 발생하므로, **Hard Negative Mining** 기법이 필수적입니다.

클래스 불균형(Class Imbalance) 해결을 위한 Negative 샘플링 전략

Ratio 3:1

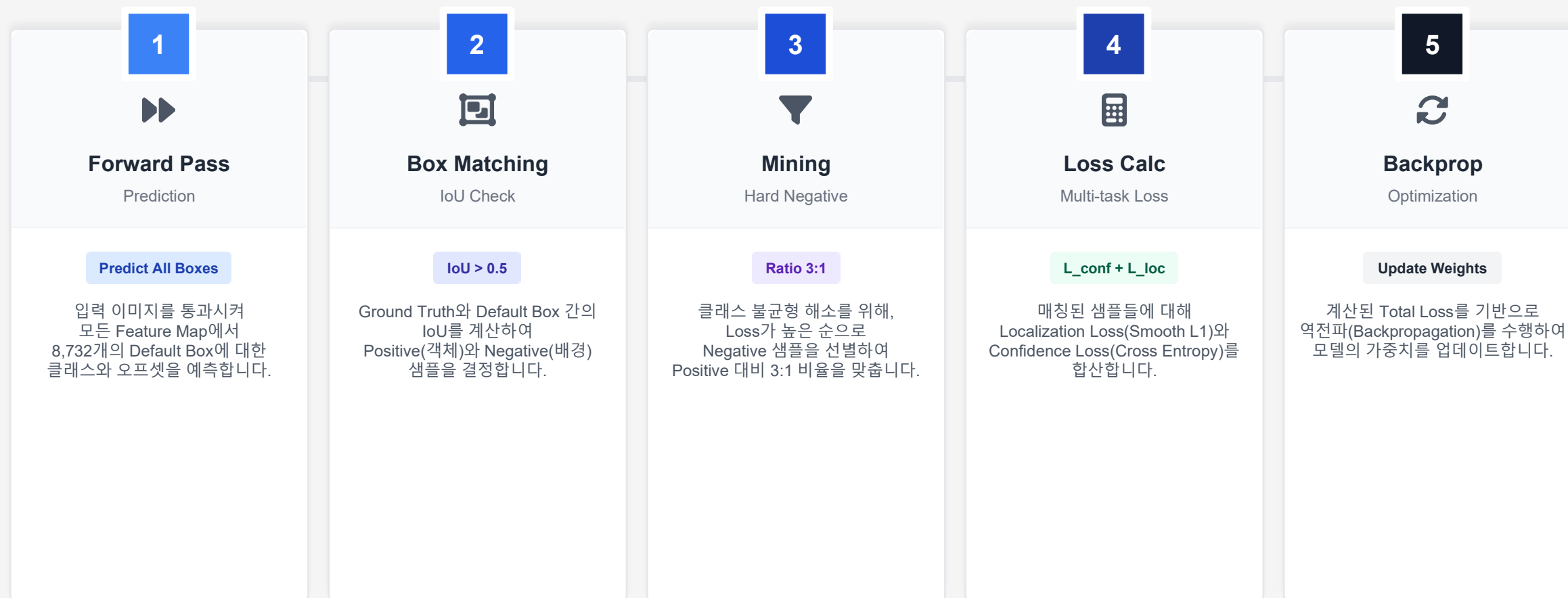
? 왜 필요한가?

이미지 내 객체(Positive)보다 배경(Negative)이 압도적으로 많습니다.
모든 배경을 학습하면 모델이 배경만 잘 맞추고 객체는 못 찾는 문제가 발생합니다.

핵심 알고리즘

- 1 모든 Negative Box의 **Confidence Loss** 계산
- 2 Loss가 높은(맞추기 어려운) 순서대로 **내림차순 정렬**
- 3 Positive 개수의 **3배**만큼 상위 Negative 선택

입력부터 가중치 업데이트까지의 5단계 학습 과정



핵심 포인트: Hard Negative Mining 단계는 배경(Background) 클래스가 압도적으로 많은 상황에서 모델이 객체 학습에 집중하도록 돕는 필수 과정입니다.

Default Box Matching 전략

Ground Truth와 Default Box의 2단계 매칭 프로세스

SSD는 학습 시 어떤 Default Box가 Ground Truth(실제 정답)에 해당하는지 결정하기 위해 **IoU(Intersection over Union)**를 기반으로 두 단계의 매칭 전략을 사용합니다.

1

Step 1

Best Prior Matching

각 Ground Truth Box에 대해, **가장 높은 IoU**를 가진 Default Box를 하나씩 우선 매칭합니다.
이는 모든 실제 객체가 최소한 하나의 Default Box와는 반드시 연결되도록 보장합니다.

2

Step 2

Threshold Matching

남은 Default Box들 중, Ground Truth와의 **IoU가 0.5 이상**인 모든 Box들을 추가로 매칭합니다.
이를 통해 하나의 객체에 대해 여러 개의 Default Box가 학습에 참여할 수 있게 되어 예측의 안정성을 높입니다.

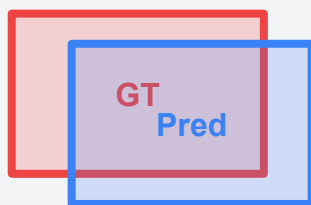
✓ 매칭 결과

위 두 과정을 통해 매칭된 Box들은 **Positive Example**(객체 있음)로 분류되어 위치/클래스 학습에 사용되고,
나머지 매칭되지 않은 Box들은 **Negative Example**(배경)로 간주됩니다.

IoU 및 매칭 전략

객체 검출의 핵심 지표 IoU와 Threshold 매칭

IoU (Intersection over Union)



Overlap Area

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{A \cap B}{A \cup B}$$

1. **Intersection (교집합, $A \cap B$):** 두 상자가 공통으로 포함하는 영역의 넓이
2. **Union (합집합, $A \cup B$):** 두 상자를 합친 전체 영역의 넓이
 1. 계산법: (상자 A의 넓이 + 상자 B의 넓이) - (교집합 넓이)

임계값 (Threshold)

0.0 0.5 (표준) 1.0



● $IoU < 0.5$: Negative (배경으로 간주)

● $IoU \geq 0.5$: Positive (매칭 성공)

</> 실제 매칭 예시

Ground Truth vs Default Box

Matched!

```
gt_box = [100, 150, 300, 400]
```

```
def_box = [95, 145, 305, 405]
```

IoU 계산 결과

```
iou_score = compute_iou(gt, def)
```

```
print(iou_score) # Result: 0.85
```

다중 매칭 (Multiple Matches)

하나의 Ground Truth에 대해 여러 개의 Default Box

✓ 목적

높은 IoU를 가진 Box들을 최대한 활용하여 학습 데이터(Positive Sample)를 확보하고, 예측 안정성을 높임

! 주의

반대로 하나의 Default Box는 오직 하나의 GT(가장 높은 IoU)에만 매칭됩니다.

SSD (Single Shot MultiBox Detector)의 강점과 한계, 실무 적용 가이드

👍 주요 강점 (Pros)

- ✓ **속도와 정확도의 균형:**
Single Stage 구조로 Real-time 처리가 가능하면서도 Multi-scale Feature Map으로 높은 정확도 달성
- ✓ **다양한 크기 대응:**
6개의 서로 다른 해상도 레이어에서 예측하여 작은 객체부터 큰 객체까지 효과적으로 검출
- ✓ **End-to-End 학습:**
Region Proposal 단계 없이 단일 네트워크로 학습 및 추론 가능

⚠ 한계점 (Cons)

- **매우 작은 객체 탐지:**
YOLO v1보다는 우수하지만, 저해상도 맵 정보 부족으로 아주 작은 물체 탐지는 여전히 도전적
- **하이퍼파라미터 의존성:**
Default Box의 스케일, 종횡비 설정이 데이터셋 특성에 크게 의존하여 튜닝 필요
- **Class Imbalance:**
많은 수의 Negative Box 생성으로 인해 Hard Negative Mining과 같은 추가 처리 필수

💡 실무 활용 팁 (Tips)

- ➔ **데이터 증강 (Augmentation):**
Zoom In/Out, Crop 등 강력한 데이터 증강이 SSD 성능 향상에 매우 결정적임
- ➔ **Custom Anchor 설계:**
K-means Clustering 등으로 데이터셋에 맞는 최적의 Aspect Ratio를 미리 분석하여 적용
- ➔ **NMS 임계값 튜닝:**
객체가 밀집된 환경에서는 NMS IoU Threshold를 신중하게 조정하여 Recall 확보

“ SSD는 속도(Fast)와 성능(Accurate)을 모두 잡은 딥러닝 객체 검출의 중요한 이정표(Milestone) 모델입니다. ”